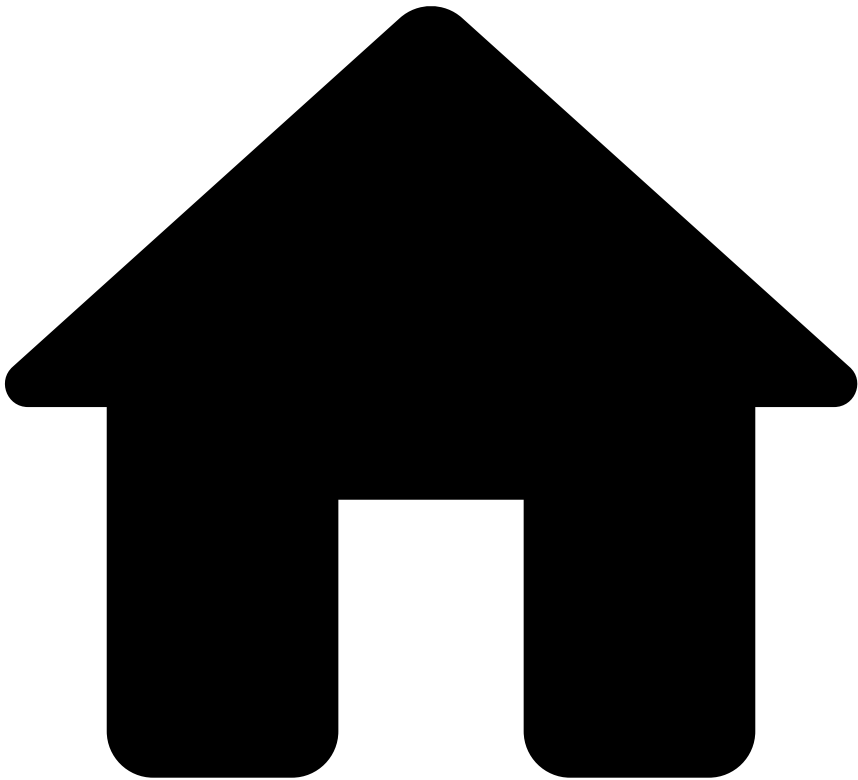


•



- [Feature Engineering](#)
- [Using PQL in Features](#)
- PQL Data Types And Conversions

On this page

PQL Data Types And Conversions

Was this page helpful? 

This topic gives you an overview of data types in PQL, and how to convert data from one data type to another.

PQL uses the following **primitive data types**.

Data type	Examples
int	-289 0

Data type	Examples
	42
long	123456789 1L -2L
float	0.99 -1.23e-100f 3.14F
double	1234567.89012 1.23456e300d 1.23456e-300d 1e1D
bool	true false
string	"John Doe" 'John Doe'

These data types are equivalent to the Java primitive data types. To see the exact definition or range for each, check the [Java documentation](#).

The **non-primitive data types** of PQL are:

- Tuples
- Collections:
 - Lists
 - Sets
 - Maps

Do not confuse these types with lists and map operators that you can create in the Pulse UI. They belong to two different levels of implementation. The non-primitive PQL data types are used only in PQL queries, when computing features. When the PQL computation is finished, Pulse stores the data of these non-primitive types as *Complex* fields in the output data source.

Data type mapping between Pulse and PQL

The following table shows to which PQL data type each [Pulse data type](#) is mapped when using input data fields:

Pulse field type (input)	PQL type
Categorical	string
Complex	the same non-primitive type of PQL

Pulse field type (input)	PQL type
Date Time	long (UNIX timestamp format)
Numeric	int, long, float, double
String	string

The following table shows which PQL data type can be passed as output to which type of output fields. The output field is the data field that the map or metric operator is creating/updating, and its type is one of the [Pulse data types](#).

PQL type	Pulse field type (output)
any data type	Categorical
int, long, float, double	Numeric
long, int (UNIX timestamp format)	Date Time
any non-primitive type	Complex
string	String

Converting data

In PQL, each data field, constant, variable, or aggregation result is of a primitive data type. To convert from one data type to another, you can use the following options:

- Cast operator, to cast a value of a numeric type into another numeric type. Learn more about this operator in the next section of this page.
- The `str()` [built-in function](#) to convert a numeric value to a string.
- [User-defined functions \(UDFs\)](#). There are [built-in UDFs](#) that can be used after creating an alias for them. Example: to convert the merchant category code to integer, you can use the `DataConversionTools.stringToInt()` function.

Cast operator

The cast operator allows you to cast values from a numeric type (int, long, float or double) to another numeric type. There is no built-in cast operator between numeric types and booleans or strings.

To cast a value, use the following syntax: `<value to convert>:<target type>`

Example: `Amount:int` is the value of the Amount field cast to integer.